香港科技大學(廣州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

UFUG 2602 LAB
Week 7: Recursion and Sorting

Tower of Hanoi, Permutations, and Insert or Merge

Ningcong Jiang
njiang300@connect.hkust-gz.edu.cn

The Hong Kong University of Science and Technology (Guangzhou)

Q1. Tower of Hanoi: Problem

Problem Description

The Tower of Hanoi game consists of three stacks (left, middle and right) and n round disks of different sizes. Initially, the left stack has all the disks, in increasing order of size from top to bottom.

The goal is to move all the disks to the right stack using the middle stack. On each move you can move the uppermost disk from a stack to another stack. In addition, it is not allowed to place a larger disk on a smaller disk.
Your task is to find a solution that minimizes the number of moves.
Follow these rules:
- Move only one disk at a time.
- Only the top disk of a rod can be moved.
- Never place a larger disk on a smaller disk.

Use recursion to find a solution with the minimum number of moves.

Input Format

One integer n, the number of disks.

Output Format

First, print the minimum number of moves.

Then, print each move on a separate line as two integers a b, meaning move the top disk from rod a to rod b.

Constraints

$1 \le n \le 10$

Hanoi: Samples and Requirements

Sample 1

```
Input
2

Output
3
1 2
1 3
2 3
```

Sample 2

```
Input
3

Output
7
1 3
1 2
3 2
1 3
2 1
2 3
1 3
```

Starter Code Requirements

Complete hanoi(n, source, auxiliary, target, moves).
n: the number of disks to move.
source: the starting rod.
auxiliary: the helper rod.
target: the destination rod.
moves: a list used to record the moves.

Record each move using
moves.append((source, target)).

The function does not need to return a value. Do not use input() or print() inside the function.
Keep the input and output code unchanged. Submit the entire program.

Idea: move n − 1 disks, move the largest disk, then move n − 1 disks again.

Hanoi: Complete Program

```
def hanoi(n, source, auxiliary, target, moves):
    if n == 1:
        moves.append((source, target))
        return
    hanoi(n - 1, source, target, auxiliary, moves)
    moves.append((source, target))
    hanoi(n - 1, auxiliary, source, target, moves)

n = int(input())
moves = []
hanoi(n, 1, 2, 3, moves)
print(len(moves))
for source, target in moves:
    print(source, target)
```

Q2. Permutations: Problem

Problem Description

Given an array nums of distinct integers, return all the possible permutations. You can return the answer in any order.

Output Format

Print all permutations, one permutation per line. Separate the integers on each line with a single space.

For this PTA version, the output must be in lexicographical order. The provided output code sorts the results for you, so your function may return the permutations in any order.

Lexicographical order compares two permutations at the first position where they differ. The permutation with the smaller integer at that position comes first.

Input Format

The first line contains an integer n, the number of elements.

The second line contains n distinct integers.

Constraints

$1 \le n \le 6$
$-10 \le nums[i] \le 10$
All integers in nums are distinct.

Implementation Requirement

Use recursion to generate the permutations. Do not use itertools.permutations().

Permutations: Samples and Requirements

Sample 1

```
Input
3
1 2 3

Output
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
```

Sample 2

```
Input
2
1 0

Output
0 1
1 0
```

Sample 3

```
Input
1
1

Output
1
```

Starter Code

Complete permute(nums) and return a list of all permutations.

Do not use input() or print() inside the function.

Keep the input and output code unchanged. Submit the entire program.

Choose an unused number.
Explore with backtrack().
Undo the choice.
Save path.copy() when full.

Permutations: Complete Program

```
def permute(nums):
    results, path = [], []
    used = [False] * len(nums)

    def backtrack():
        if len(path) == len(nums):
            results.append(path.copy())
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            used[i] = True
            path.append(nums[i])
            backtrack()
            path.pop()
            used[i] = False
    backtrack()
    return results

n = int(input())
nums = list(map(int, input().split()))
results = permute(nums)
results.sort()
for permutation in results:
    print(*permutation)
```

Q3. Insert or Merge: Problem

Problem Description

According to Wikipedia:
Insertion sort iterates, consuming one input element each repetition, and growing a sorted output list. Each iteration, insertion sort removes one element from the input data, finds the location it belongs within the sorted list, and inserts it there. It repeats until no input elements remain.
Merge sort works as follows: Divide the unsorted list into N sublists, each containing 1 element (a list of 1 element is considered sorted). Then repeatedly merge two adjacent sublists to produce new sorted sublists until there is only 1 sublist remaining.
Now given the initial sequence of integers, together with a sequence which is a result of several iterations of some sorting method, can you tell which method we are using?

Input Specification

Each input file contains one test case. For each case, the first line gives a positive integer N (< 100). Then in the next line, N integers are given as the initial sequence. The last line contains the partially sorted sequence of the N numbers. It is assumed that the target sequence is always ascending. All the numbers in a line are separated by a space.

Insert or Merge: Output and Samples

Output Specification

For each test case, print in the first line either "Insertion Sort" or "Merge Sort" to indicate the method used to obtain the partial result. Then run this method for one more iteration and output in the second line the resulting sequence. It is guaranteed that the answer is unique for each case. All the numbers in a line must be separated by a space, and there must be no extra space at the end of the line.

Sample 1

```
Input
10
3 1 2 8 7 5 9 4 6 0
1 2 3 7 8 5 9 4 6 0

Output
Insertion Sort
1 2 3 5 7 8 9 4 6 0
```

Sample 2

```
Input
10
3 1 2 8 7 5 9 4 6 0
1 3 2 8 5 7 4 9 0 6

Output
Merge Sort
1 2 3 8 4 5 7 9 0 6
```

Insert or Merge: Complete Program

```
def insert_or_merge(a, b):
    n = len(a)
    p = 1
    while p < n and b[p - 1] <= b[p]:
        p += 1
    if b[p:] == a[p:]:
        b = b.copy()
        if p < n:
            key, j = b[p], p - 1
            while j >= 0 and b[j] > key:
                b[j + 1] = b[j]
                j -= 1
            b[j + 1] = key
        return "Insertion Sort", b
    size = 1
    while size < n:
        a = merge_pass(a, size)
        size *= 2
        if a == b:
            a = merge_pass(a, size)
            return "Merge Sort", a
```

```
def merge_pass(a, size):
    result = []
    for start in range(0, len(a), 2 * size):
        left = a[start:start + size]
        end = start + 2 * size
        right = a[start + size:end]
        i, j = 0, 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                result.append(left[i])
                i += 1
            else:
                result.append(right[j])
                j += 1
        result.extend(left[i:])
        result.extend(right[j:])
    return result
# Input, process, output.
n = int(input())
a = list(map(int, input().split()))
b = list(map(int, input().split()))
method, result = insert_or_merge(a, b)
print(method)
print(*result)
```